

Avant de joindre

Comprendre la cardinalité

Avant chaque jointure, identifier la relation entre les deux tables :

Relation	Signification	Comportement attendu
1-1	Chaque clé est unique des deux côtés	Nombre de lignes inchangé
1-N	La clé est unique dans une table, multiple dans l'autre	Nombre de lignes = côté N
N-N	La clé est multiple des deux côtés	Explosion du volume — à éviter

Une jointure N-N inattendue multiplie les lignes silencieusement sans lever d'erreur. C'est la source de bug la plus difficile à détecter.

Vérifier les types

Les colonnes de jointure doivent avoir exactement le même type des deux côtés. Une jointure **string** vs **int** ne lève pas d'erreur mais ne matche aucune ligne — le résultat est vide sans avertissement.

Sélectionner les colonnes utiles en amont

Ne garder que les colonnes nécessaires dans chaque table **avant** de joindre. Transporter 40 colonnes pour n'en utiliser que 3 gaspille de la mémoire et ralentit les opérations suivantes.

Choisir le bon type de jointure

Type	Quand l'utiliser
inner	La relation est obligatoire — une ligne sans correspondance n'a pas de sens
left	On veut garder toutes les lignes de gauche, la correspondance est optionnelle
outer	On veut conserver les lignes des deux tables même sans correspondance

Ne pas utiliser **left** par défaut parce que c'est "plus sûr" — si la relation est obligatoire, utiliser **inner** et traiter les lignes sans correspondance comme des anomalies à investiguer.

Après la jointure

Vérifier le volume

Contrôler que le nombre de lignes en sortie correspond à ce qui est attendu selon la cardinalité identifiée. Un volume supérieur à ce qui est attendu signale presque toujours une relation N-N non anticipée.

Gérer les colonnes en collision

Quand les deux tables ont des colonnes de même nom (hors clé de jointure), Pandas génère automatiquement des suffixes **_x** et **_y**. Ne jamais laisser ces suffixes dans le dataset final — renommer ou supprimer explicitement avant d'écrire le résultat.

Gérer les valeurs nulles introduites

Un **left join** introduit des nulls sur toutes les colonnes de la table de droite quand il n'y a pas de correspondance. Décider explicitement du comportement : conserver les nulls, les remplir avec une valeur par défaut, ou filtrer les lignes sans correspondance.

Jointures en cascade

Quand plusieurs jointures sont enchaînées sur le même DataFrame, vérifier le volume **après chaque étape**, pas seulement à la fin. Une multiplication de lignes au premier merge se propage et s'amplifie à chaque jointure suivante.

Récapitulatif — Ce qu'il ne faut jamais faire

À éviter	Raison
Joindre sans avoir vérifié la cardinalité	Multiplication silencieuse des lignes
Joindre sur des colonnes de types différents	Ne matche rien, sans erreur
Utiliser <code>left</code> par défaut sans réfléchir	Masque les données manquantes qui sont des
Laisser les suffixes <code>_x</code> / <code>_y</code> dans le résultat	Colonnes ambiguës, bugs silencieux en aval
Joindre sur une colonne calculée à la volée	Calculer la colonne avant et stocker le résultat
Transporter toutes les colonnes avant de	Gaspillage mémoire, ralentissement des opérations
Ne vérifier le volume qu'à la fin d'une	Impossible de localiser quelle jointure a multiplié

explique relation N-N avec un exemple et est ce que c'est le produit cartésien ?

Relation N-N et produit cartésien

C'est quoi une relation N-N ?

Une relation N-N signifie que la clé de jointure apparaît **plusieurs fois des deux côtés** de la jointure.

Exemple concret

Imagine deux tables du projet COR :

Table A — Événements (plusieurs événements par contrat)

CONTRACT	event_date	type
C001	2024-01-01	EPU
C001	2024-02-01	EPG
C001	2024-03-01	EPU

Table B — Marchés (plusieurs marchés par contrat — cas anormal)

CONTRACT	marche
C001	MARCHE_A
C001	MARCHE_B

Ce qui se passe avec un join N-N

```
df_result = df_event.merge(df_marche, on="CONTRACT", how="left")
```

Résultat obtenu — 6 lignes au lieu de 3 :

CONTRACT	event_date	type	marche
C001	2024-01-01	EPU	MARCHE_A
C001	2024-01-01	EPU	MARCHE_B
C001	2024-02-01	EPG	MARCHE_A
C001	2024-02-01	EPG	MARCHE_B
C001	2024-03-01	EPU	MARCHE_A
C001	2024-03-01	EPU	MARCHE_B

Chaque ligne de A est combinée avec **chaque** ligne de B qui partage la même clé.

Est-ce que c'est un produit cartésien ?

Oui et non — c'est un produit cartésien local.

	Produit cartésien complet	Jointure N-N
Définit	Toutes les lignes de A × toutes les	Toutes les lignes de A × toutes les lignes de B qui
Condi	Aucune	Sur une clé commune
Volume	$N_A \times N_B$	Dépend du nombre de doublons par clé

Sur l'exemple : le contrat C001 a 3 événements et 2 marchés → $3 \times 2 = 6$ lignes pour ce contrat.

C'est exactement un produit cartésien, mais limité aux lignes qui partagent la même valeur de clé.

Si tous les contrats avaient 2 doublons dans la table marché, le volume total doublerait. Si certains en ont 10, ces contrats-là auraient 10× plus de lignes — et le dataset final aurait un volume incohérent sans aucun message d'erreur.

Pourquoi c'est dangereux

- Aucune erreur n'est levée — le code s'exécute normalement
- Le volume augmente silencieusement
- Les calculs d'agrégation en aval (sommes, moyennes) donnent des résultats faux car les lignes sont comptées plusieurs fois
- Le bug est difficile à détecter si on ne vérifie pas le nombre de lignes après la jointure

Comment s'en protéger

Avant de joindre, vérifier que la table de référence est bien dédoublée sur la clé :

```
# Vérifier qu'il n'y a pas de doublons sur la clé de jointure
assert df_marche["CONTRACT"].is_unique, "Doublons sur CONTRACT dans df_marche"
```

Ou dédupliquer explicitement avant le merge si on sait que des doublons peuvent exister et qu'on veut garder une seule ligne :

```
df_marche = df_marche.drop_duplicates(subset=["CONTRACT"])
```